



You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

Add a Built-in Template

This guide is for **developers** who want to add new built-in templates to Noctalia.

For **users** who want to create custom templates for their own applications, see [User Templates](#).

Overview

Adding a new built-in template involves two steps:

1. [Create the template file](#)
 2. [Register the template](#)
-

Step 1: Create the Template File

Create your template file in `Assets/Templates/`:

`Assets/Templates/myapp.conf`

```
background = {{colors.surface.default.hex}}
foreground = {{colors.on_surface.default.hex}}
accent = {{colors.primary.default.hex}}
border = {{colors.outline.default.hex}}
```

For complete template syntax documentation (colors, formats, filters, blocks), see the [User Templates](#) guide.

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

Step 2: Register the Template

Open **Services/Theming/TemplateRegistry.qml** and add your template.

For Terminal Emulators

Add to the terminals array:

```
{
  "id": "kitty",
  "name": "Kitty",
  "templatePath": "terminal/kitty.conf",
  "outputPath": "~/.config/kitty/themes/noctalia.conf",
  "postHook": `${templateApplyScript} kitty`
}
```

Properties:

- **id** - Unique identifier used in settings
- **name** - Display name in the UI
- **templatePath** - Relative path from Assets/Templates/
- **outputPath** - Destination path (~ is expanded to home directory)
- **postHook** - Shell command to run after generation (e.g., to reload the terminal)

For Applications

Add to the applications array:

```
{  
  "id": "myapp",
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
  "category": "misc",  
  "input": "myapp.conf",  
  "outputs": [  
    {  
      "path": "~/.config/myapp/themes/noctalia.conf"  
    }  
  ],  
  "postProcess": (mode) ⇒ `${templateApplyScript} myapp`  
}
```

Properties:

- **id** - Unique identifier used in settings
- **name** - Display name in the UI
- **category** - Category for organization (see below)
- **input** - Template filename in Assets/Templates/
- **outputs** - Array of output destinations, each with a path property
- **postProcess** - Optional function receiving the current mode ("dark" or "light")

Advanced output options:

```
"outputs": [  
  {  
    "path": "~/.config/myapp/themes/noctalia.conf",  
    "input": "myapp-alternate.conf" // Override input for this output  
  }  
]
```

Additional flags:

- **dualMode: true** - Template contains both dark and light theme patterns (e.g., zed.json)
- **strict: true** - Use strict mode for palette generation (preserves custom surface/outline values)

Once registered:

1. The template will automatically appear in **Settings > Color Scheme > Templates**.
You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)
2. The UI will automatically show the target path in the tooltip.
3. The state (enabled/disabled) is automatically saved.

Multi-Client Applications

Some applications have multiple variants (e.g., Discord forks, VS Code variants). Use the clients array pattern:

```
{
  "id": "discord",
  "name": "Discord",
  "category": "misc",
  "input": ["discord-midnight.css", "discord-material.css"],
  "clients": [
    { "name": "vesktop", "path": "~/config/vesktop" },
    { "name": "webcord", "path": "~/config/webcord" },
    { "name": "equibop", "path": "~/config/equibop" }
  ]
}
```

For multi-client apps:

- **input** can be an array of template files (generates multiple themes per client)
- **clients** array defines the different application variants
- Each client has a name (identifier) and path (config directory)
- Clients are auto-detected by ProgramCheckerService - only detected clients receive themes

Existing multi-client apps:

- **Discord** - vesktop, webcord, armcord, equibop, equicord, lightcord, dorion, vencord, betterdiscord
- **Code** - code (VS Code), codium (VSCodium)

- **Emacs** - doom, modern (~~/config/emacs~~), traditional (~~/etc/emacs.d~~)

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

Template Categories

Use the category property to organize your template in the settings UI:

Category	Examples
system	GTK, Qt, KDE
terminal	Automatically assigned for terminals array
launcher	App launchers
browser	Web browsers
editor	Code editors
audio	Music players, visualizers
compositor	Hyprrland, Niri, etc.
misc	Everything else

How Color Generation Works

Noctalia uses `template-processor.py` to handle all color generation and template rendering.

1. Wallpaper Analysis

When “Wallpaper Colors” is enabled, the processor analyzes the current wallpaper image using:

- **Quantization:** Extracts dominant colors using the Wu quantizer (same as Android/Matugen)
- **Scoring:** Ranks colors based on chroma and population to find the best seed color

- **Palette Generation:** Creates a full palette from one or more seed colors

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

When using a predefined scheme (e.g., Nord, Dracula), the processor:

- Loads the scheme’s core colors
- Generates the missing Material Design 3 tokens to ensure a complete palette

3. Rendering

The processor parses `{{...}}` tags, applies any filters, and writes the output files.

Scheme Types

When using wallpaper colors, different palette generation algorithms are available:

Scheme Type	Description
M3-Tonal Spot	Default Material Design 3 algorithm with tonal variations
M3-Content	Emphasizes content colors from the source
M3-Fruit Salad	More vibrant, varied color selection
M3-Rainbow	Full spectrum rainbow-based palette
M3-Monochrome	Single-hue variations for minimalist themes
Vibrant	Maximizes saturation for bold colors
Faithful	Stays closer to source image colors
Muted	Desaturated, subtle color palette

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)
